

Security Operations & Detection Engineering Platform



Part 1: Telemetry Collection, Attack Simulation & Threat Investigation

By Miguel K.

Introduction

Modern security operations rely on centralized visibility, detection engineering, and investigative capabilities to identify and respond to evolving cyber threats. To better understand these processes, I designed and built a Security Operations & Detection Engineering Platform capable of collecting, correlating, and analyzing security telemetry across Windows and Linux environments through a centralized SIEM.

Leveraging Splunk Enterprise, Sysmon, Windows, Kali Linux, and Ubuntu Server, I developed a multi-platform environment to simulate attacker behavior, validate telemetry pipelines, and investigate security events from a defender's perspective. Adversary simulations including network reconnaissance, SSH brute-force attacks, HTTP beaconing, and LOLBin abuse were executed to generate telemetry for monitoring, detection, and investigation.

As Part 1 of a larger initiative, this project focused on building the foundational components required for effective security operations, including telemetry collection, log ingestion, attack simulation, detection validation, and investigation workflows. These capabilities establish the groundwork for future phases involving advanced detection engineering, threat hunting, automated response workflows, webhooks, SOAR integrations, and AI-driven security operations.

This presentation highlights the architecture, telemetry pipelines, attack simulations, detection capabilities, and investigation workflows developed throughout the project.

Environment Overview

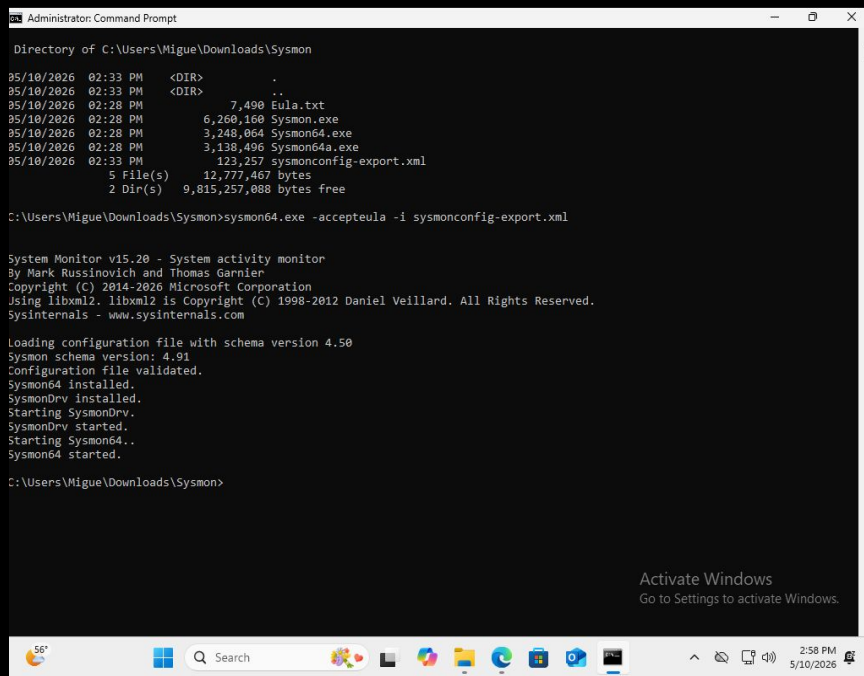
The Security Operations & Detection Engineering Platform was built using a multi-system architecture designed to simulate both attacker and defender workflows.

Core Components

- Windows 11 Monitored Endpoint
- Kali Linux Adversary System
- Ubuntu Server
- Splunk Enterprise
- Sysmon
- Splunk Universal Forwarder

Primary Objectives

- Generate attack telemetry
 - Centralized log collection
 - Validate detections and investigations
 - Simulate real-world SOC operations
-



```
Administrator Command Prompt

Directory of C:\Users\Migue\Downloads\Sysmon

35/10/2026  02:33 PM    <DIR>          .
35/10/2026  02:33 PM    <DIR>          ..
35/10/2026  02:28 PM                7,490 Eula.txt
35/10/2026  02:28 PM        6,260,160 Sysmon.exe
35/10/2026  02:28 PM        3,248,064 Sysmon64.exe
35/10/2026  02:28 PM        3,138,496 Sysmon64a.exe
35/10/2026  02:33 PM         123,257 sysmonconfig-export.xml
35/10/2026  02:33 PM            5 File(s)      12,777,467 bytes
                2 Dir(s)   9,815,257,088 bytes free

C:\Users\Migue\Downloads\Sysmon>sysmon64.exe -accepteula -i sysmonconfig-export.xml

System Monitor v15.20 - System activity monitor
By Mark Russinovich and Thomas Garnier
Copyright (c) 2014-2026 Microsoft Corporation
Using libxml2. libxml2 is Copyright (C) 1998-2012 Daniel Veillard. All Rights Reserved.
Sysinternals - www.sysinternals.com

Loading configuration file with schema version 4.50
Sysmon schema version: 4.91
Configuration file validated.
Sysmon64 installed.
SysmonDrv installed.
Starting SysmonDrv.
SysmonDrv started.
Starting Sysmon64..
Sysmon64 started.

C:\Users\Migue\Downloads\Sysmon>
```

To establish advanced endpoint visibility within the environment, Sysmon was deployed on the Windows 11 monitored endpoint.

Sysmon was installed using the SwiftOnSecurity configuration to enhance telemetry collection and provide greater visibility into endpoint activity.

Sysmon Telemetry Deployment

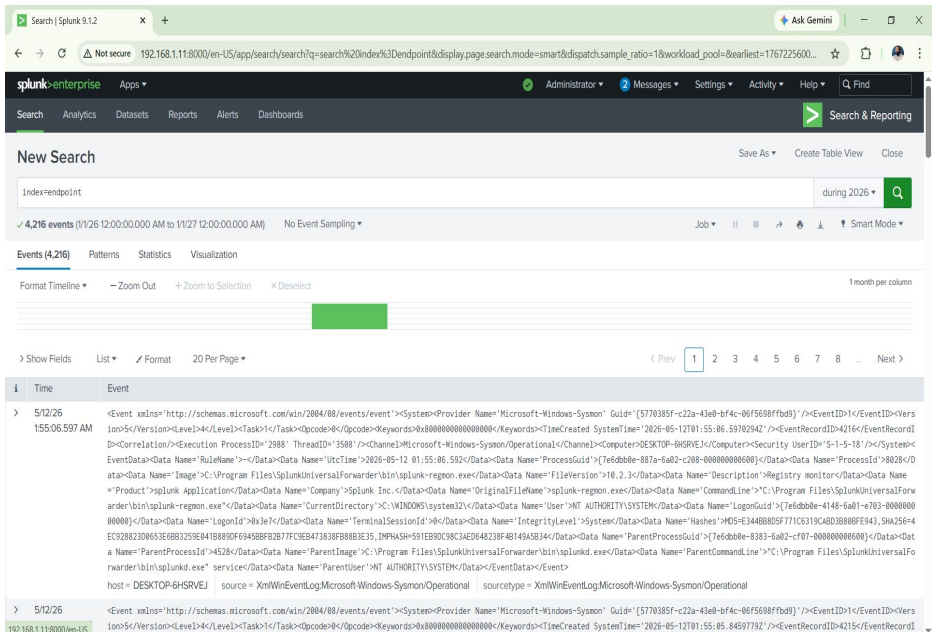
The configuration enables detailed logging of security-relevant events including:

- Process creation
- Network connections
- PowerShell activity
- Driver loading
- File creation activity
- Parent-child process relationships
- Executable hash generation

The deployment of Sysmon significantly expanded endpoint telemetry and provided the event data required for:

- Threat detection
- Detection engineering
- Threat hunting
- Security monitoring
- Incident response analysis
- Adversary activity visibility within Splunk Enterprise

This phase established the core telemetry foundation for future attack simulations and detection validation activities.



Following the deployment of Sysmon and the Splunk Universal Forwarder on the Windows 11 endpoint, endpoint telemetry was successfully forwarded into Splunk Enterprise for centralized log ingestion and analysis.

A dedicated Splunk index was created to store endpoint telemetry generated by Sysmon, and the Universal Forwarder was configured to ingest the Microsoft-Windows-Sysmon operational event channel into the SIEM environment.

Sysmon Telemetry Ingestion into Splunk

Following forwarder configuration and service-level permission adjustments, Sysmon telemetry began successfully populating within Splunk Search & Reporting, confirming operational log ingestion between the monitored endpoint and the SIEM platform.

This phase established the foundational telemetry pipeline required for:

- Threat detection
- Detection engineering
- Threat hunting
- Endpoint monitoring
- Adversary activity visibility
- Incident response analysis

The environment is now capable of ingesting and analyzing real endpoint telemetry generated from future attack simulation and offensive security activities performed within the lab.

Splunk Alerting & Detection Engineering

Scheduled Splunk alerts were developed and validated to detect suspicious PowerShell activity within the environment.

Detection content focused on PowerShell-based attack behaviors including: **Malicious scripting activity - Obfuscation techniques - Defense evasion - Suspicious PowerShell execution patterns**

Implemented detections included: **Encoded PowerShell Execution - ExecutionPolicy Bypass Activity**

The detections were mapped to: **MITRE ATT&CK T1059.001 – PowerShell**

Adversary simulations were executed on the Windows 11 monitored endpoint using attacker-style PowerShell techniques. Generated telemetry was captured by Sysmon and forwarded to Splunk Enterprise through the Splunk Universal Forwarder.

Custom alerts were configured to: **Monitor suspicious PowerShell execution - Identify encoded command usage - Detect ExecutionPolicy bypass behavior - Generate alerts for suspicious activity**

Alert validation confirmed: **Telemetry ingestion - Detection visibility - Alert triggering - ATT&CK-aligned detection engineering - SIEM monitoring functionality**

This phase expanded the environment from passive telemetry collection to proactive detection and alerting. **The following slide demonstrates the detection logic, alert validation, and monitoring capabilities developed during this phase.**



(continued)

Splunk Alerting & Detection Engineering (continued)

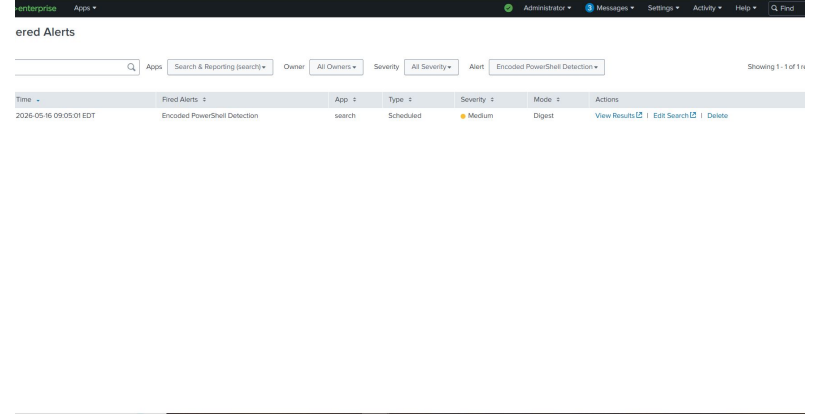
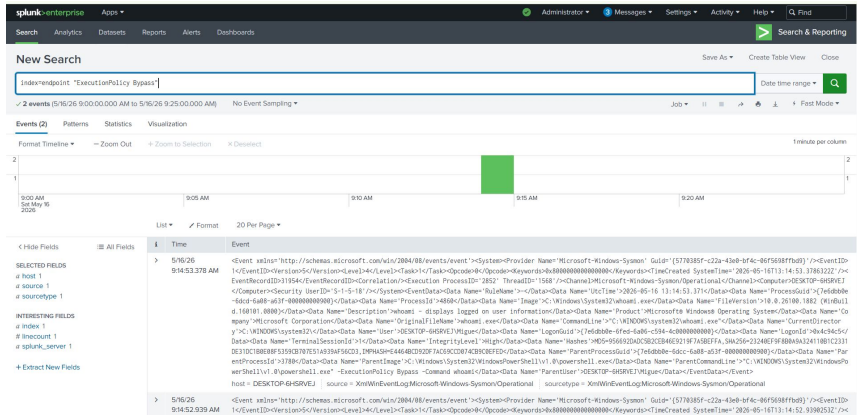


Figure 1: Splunk detection PowerShell ExecutionPolicy Bypass activity.

Figure 2: Successful alert generation following detection of suspicious PowerShell activity.

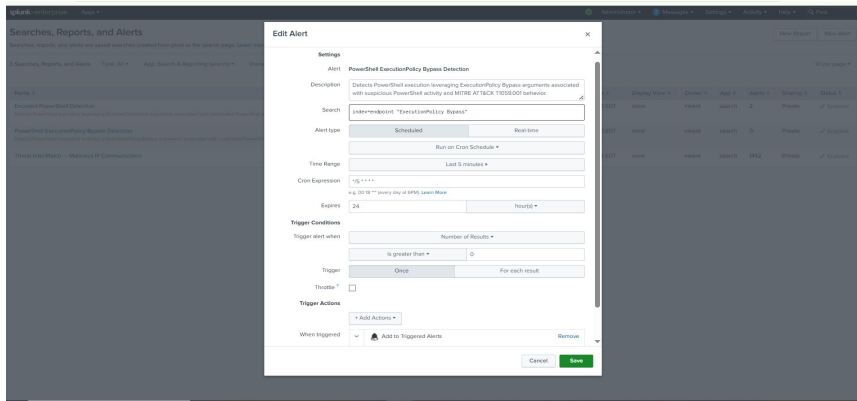


Figure 3: Scheduled Splunk alert configured to monitor and detect PowerShell-based attack activity.

Telemetry Validation & Reconnaissance Traffic Generation

Reconnaissance telemetry and network visibility were validated across the lab environment through the generation of attacker-originated traffic from the Kali Linux VM toward monitored systems.

Technologies utilized during this activity included: **Splunk Enterprise, Sysmon, Kali Linux, Suricata, tcpdump, VMware Workstation, Windows 11, and Ubuntu Server.**

Simulated reconnaissance activity included Nmap scanning and HTTP-based interactions designed to generate observable network telemetry and validate end-to-end visibility across the environment.

Network monitoring capabilities were validated using Suricata and tcpdump, while a temporary listening service was created on the Windows endpoint to support telemetry generation and traffic analysis.

Validation activities included **restoring monitored packet visibility through VMware bridged networking, confirming attacker-to-target communication, generating reconnaissance traffic, and validating the successful ingestion of attacker-originated telemetry into Splunk Enterprise.**

Analysis of the generated telemetry confirmed visibility into **source IP addresses, destination IP addresses, destination ports, and attacker-attributed network activity** across the monitored environment.

This activity successfully validated network visibility, telemetry collection, and SIEM ingestion capabilities while demonstrating the ability to monitor, analyze, and attribute reconnaissance activity within the environment.



(continued)

Telemetry Validation & Reconnaissance Traffic Generation (continued)

```
and Greppable format, respectively, to the given filename.
-oA <filename>: Output in the three major formats at once
-v: Increase verbosity level (use -vv or more for greater effect)
-d: Increase debugging level (use -dd or more for greater effect)
--reason: Display the reason a port is in a particular state
--open: Only show open (or possibly open) ports
--packet-trace: Show all packets sent and received
--iflist: Print host interfaces and routes (for debugging)
--append-output: Append to rather than clobber specified output files
--resume <filename>: Resume an aborted scan
--noninteractive: Disable runtime interactions via keyboard
--stylesheet <path/url>: XSL stylesheet to transform XML output to HTML
--webxml: Reference stylesheet from Nmap.org for more portable XML
--no-stylesheet: Prevent associating of XSL stylesheet w/XML output
MISC:
-G: Enable IPv6 scanning
-A: Enable OS detection, version detection, script scanning, and traceroute
--datadir <dirname>: Specify custom Nmap data file location
--send-eth/--send-ip: Send using raw ethernet frames or IP packets
--privileged: Assume that the user is fully privileged
--unprivileged: Assume the user lacks raw socket privileges
-V: Print version number
-hi: Print this help summary page.
EXAMPLES:
nmap -v -A scanme.nmap.org
nmap -sn 192.168.0.0/16 10.0.0.0/8
nmap -v -IR 10000 -pn -p 80
SEE THE MAN PAGE (https://nmap.org/book/man.html) FOR MORE OPTIONS AND EXAMPLES

(miguel@spc-kali) ~
$ nmap -pn -p 8080 192.168.1.60
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-17 11:03 -0400
Nmap scan report for 192.168.1.60
Host is up (0.40052s latency).

PORT      STATE SERVICE
8080/tcp  open  http-proxy
MAC Address: 68:54:5A:60:C5:DE (Intel Corporate)
```

Figure 1: Successful reconnaissance scan from the attacker VM detecting the exposed TCP 8080 service on the Windows endpoint.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\WINDOWS\system32> $listener = [System.Net.Sockets.TcpListener]8080; $listener.Start(); while($true){$cli
ent=$listener.AcceptTcpClient()}
```

Figure 2: Temporary PowerShell-based TCP listening service configured on the Windows endpoint for telemetry and reconnaissance validation.

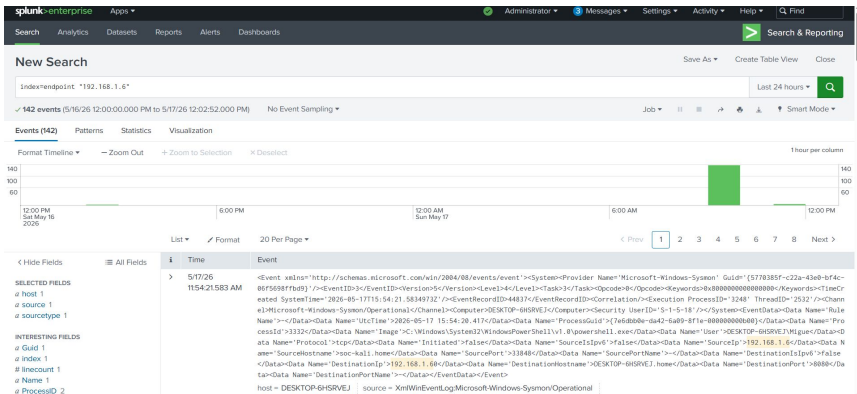


Figure 3: Splunk telemetry showing attacker-originated network activity including source IP, destination IP, and destination port visibility.

Detection Engineering & Telemetry Analysis

Endpoint telemetry generated from the Windows environment was analyzed and validated within Splunk using Sysmon event logging. The focus of this activity was **process creation visibility, network connection telemetry, and behavioral analysis**.

Sysmon event ingestion was validated through the investigation of Event ID 1 process creation events and Event ID 3 network connection events. Analysis confirmed visibility into **command-line execution, process lineage, hash collection, user context, source and destination IP addresses, and destination port activity**.

Telemetry analysis successfully identified attacker-originated network activity from the Kali Linux system and demonstrated the ability to correlate endpoint and network-based events within Splunk.

Behavioral detection searches were developed using Sysmon telemetry, while initial alerting logic and dashboard components were created to support future monitoring and analysis efforts.

This activity successfully validated Sysmon telemetry ingestion, endpoint visibility, and foundational detection engineering capabilities within the environment.

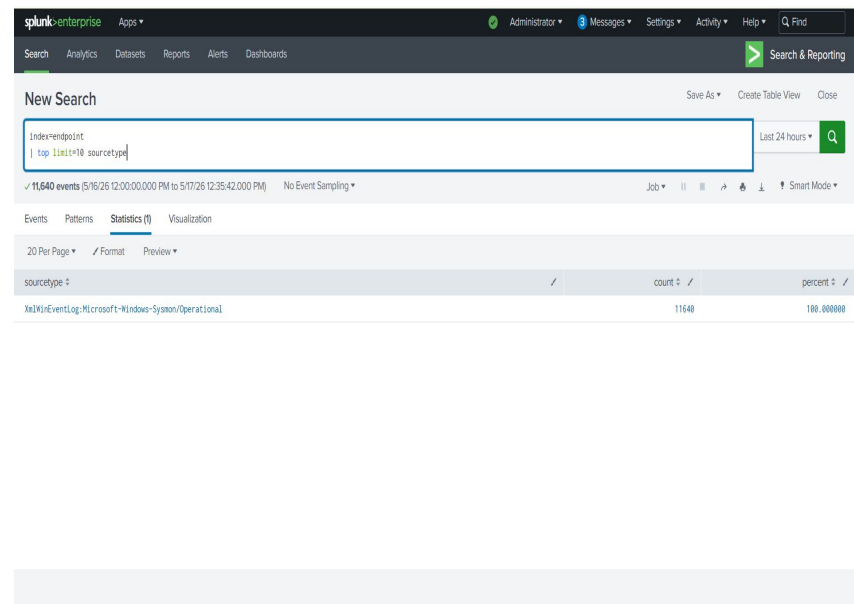


Figure 1: Displays the primary telemetry sources successfully ingested into Splunk Enterprise, validating Sysmon event collection and endpoint telemetry visibility.

➡ (continued)

Detection Engineering & Telemetry Analysis (continued)

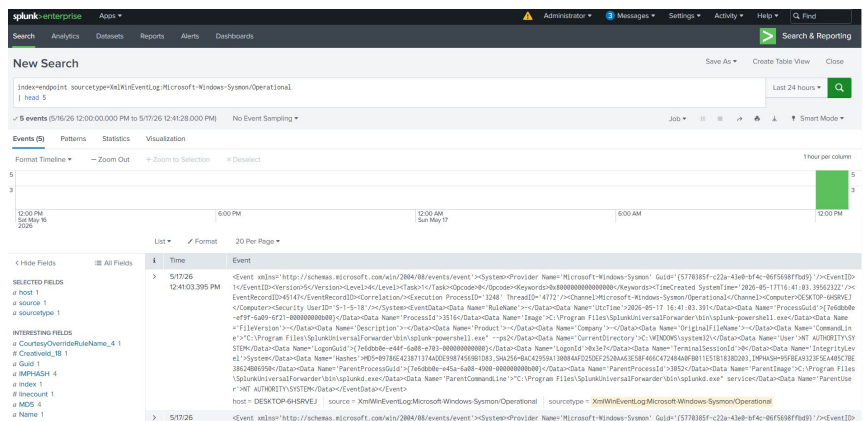


Figure 2: Shows raw Sysmon event data inside Splunk, confirming successful ingestion and parsing of Windows endpoint telemetry.

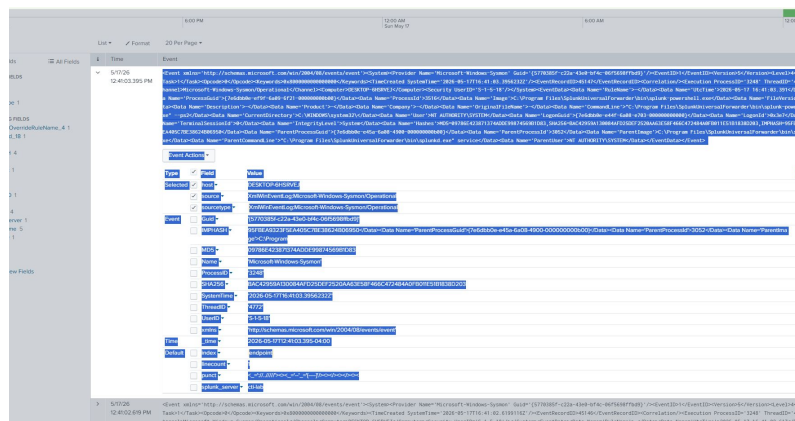
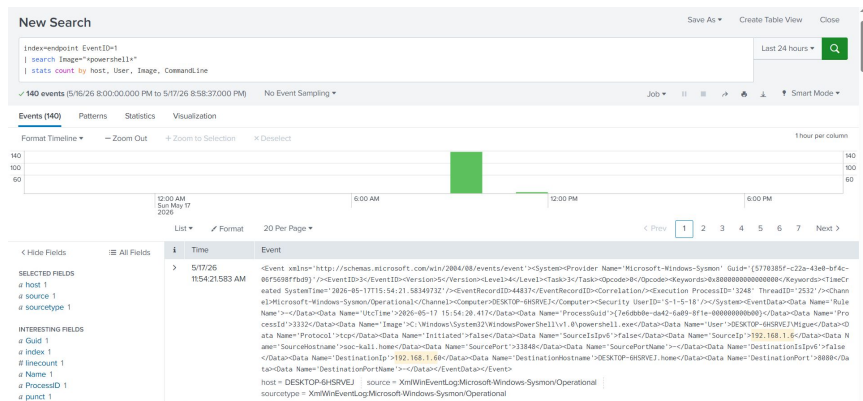
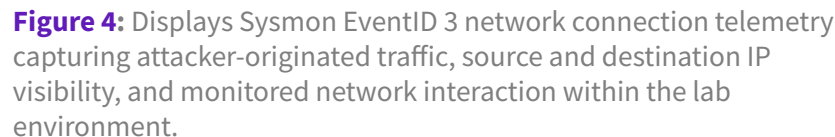


Figure 3: Demonstrates Sysmon EventID 1 process creation telemetry, including command-line visibility, process lineage, user context, and hash collection for behavioral analysis.



Adversary Simulation, Telemetry Validation & Investigation

This phase focused on simulating adversary behavior across Linux and Windows systems while validating telemetry collection and visibility within Splunk.

Attack simulations included **network reconnaissance using Nmap, SSH brute-force activity using Hydra, HTTP beaconing, and LOLBin abuse through Certutil**. Each technique generated authentication, network, application, and endpoint telemetry that could be analyzed from a defender's perspective.

Following the attack simulations, an investigation was conducted within Splunk using a suspicious source IP address as the initial indicator. Telemetry was correlated across Linux authentication logs, custom beaconing logs, and Windows Sysmon events to identify and validate attacker activity.

The investigation demonstrated how a single indicator can be used to pivot across multiple telemetry sources and build a broader understanding of attacker behavior.

This activity successfully validated attack simulation, telemetry collection, log ingestion, and investigation workflows while demonstrating end-to-end visibility across the environment.

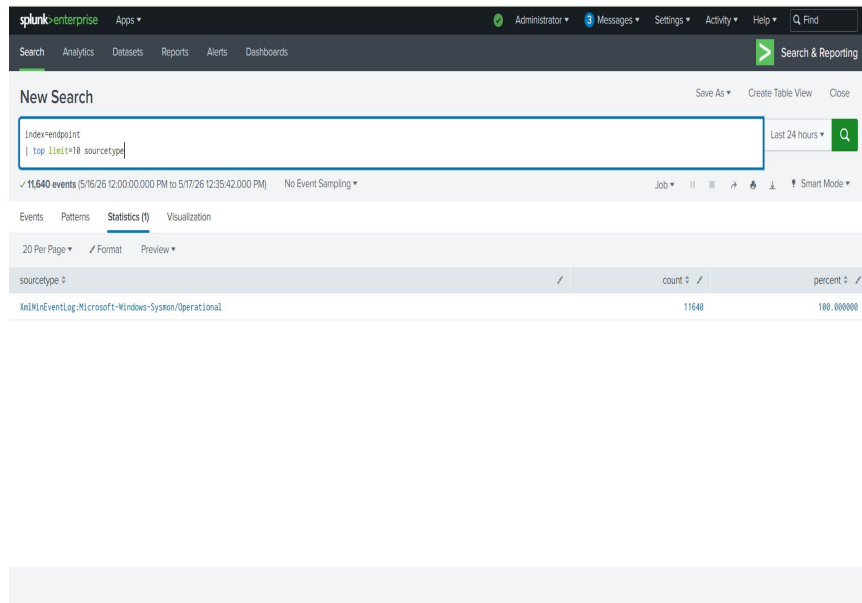


Figure 1: Displays the primary telemetry sources successfully ingested into Splunk Enterprise, validating Sysmon event collection and endpoint telemetry visibility.



(continued)

Adversary Simulation, Telemetry Validation & Investigation (continued)

```

--append-output: Append to rather than clobber specified output files
--resume <filename>: Resume an aborted scan
--noninteractive: Disable runtime interactions via keyboard
--stylesheet <path/url>: XSL stylesheet to transform XML output to HTML
--webxml: Reference stylesheet from Nmap.Org for more portable XML
--no-stylesheet: Prevent associating of XSL stylesheet w/XML output

NMAP:
--s: Enable IPv6 scanning
--A: Enable OS detection, version detection, script scanning, and traceroute
--datadir <dirname>: Specify custom Nmap data file location
--send-eth/--send-ip: Send using raw ethernet frames or IP packets
--privileged: Assume that the user is fully privileged
--unprivileged: Assume the user lacks raw socket privileges
--v: Print version number
--h: Print this help summary page.

EXAMPLES:
nmap -v -A scanme.nmap.org
nmap -sn 192.168.0.0/16 10.0.0.0/8
nmap -v -IR 10000 -Pn -p 80
SEE THE MAN PAGE (https://nmap.org/book/man.html) FOR MORE OPTIONS AND EXAMPLES

[miguels@sec-kali]~$ nmap -sV -Pn 192.168.1.60
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-20 20:02 -0400
Nmap scan report for DESKTOP-1LGTW87-home (192.168.1.60)
Host is up (0.00051s latency).
Not shown: 993 filtered tcp ports (no-response)
PORT      STATE SERVICE
135/tcp   open  msrpc      Microsoft Windows RPC
139/tcp   open  netbios-ssn Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds
2869/tcp  open  http       Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
3257/tcp  open  http       Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
7070/tcp  open  ssl/realserver
8000/tcp  open  http-proxy
MAC Address: 68:5A:5A:68:C5:D6 (Intel Corporate)
Service Info: OS: windows; CPE: cpe:/o:microsoft:windows

```

Figure 2: Nmap reconnaissance activity performed against the Ubuntu target to identify accessible services and validate communication between attacker and target systems.

```

Session Actions Edit View Help
[miguels@sec-kali]~$ hydra -l test -P /usr/share/wordlists/rockyou.txt ssh://192.168.1.50 -t 4 -f
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is no
n-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-20 22:07:15
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.rest
ore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l1/p1:14344399), ~3586100 tries per task
[DATA] attacking ssh://192.168.1.50:22/
[STATUS] 64.00 tries/min, 64 tries in 00:01h, 14344335 to do in 3735:31h, 4 active
[STATUS] 64.07 tries/min, 240 tries in 00:02h, 14344399 to do in 3508:02h, 4 active
^CThe session file ./hydra.restore was written. Type "hydra -R" to resume session.

[miguels@sec-kali]~$
[miguels@sec-kali]~$ hydra -l test -P /usr/share/wordlists/rockyou.txt ssh://192.168.1.50 -t 4 -f
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is no
n-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-22 11:59:47
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.rest
ore
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l1/p1:14344399), ~3586100 tries per task
[DATA] attacking ssh://192.168.1.50:22/
[STATUS] 64.00 tries/min, 64 tries in 00:01h, 14344335 to do in 3735:31h, 4 active
[STATUS] 64.07 tries/min, 240 tries in 00:02h, 14344399 to do in 3401:37h, 4 active
^CThe session file ./hydra.restore was written. Type "hydra -R" to resume session.

[miguels@sec-kali]~$

```

Figure 3: Controlled SSH brute-force attack executed using Hydra against the Ubuntu target to generate relevant credential attack telemetry.

Not Secure 192.168.1.50(8000) en-US/app/search/search?q=search%20order%3Ddesc%20source%3Dnmap_securedisplaypage=searchmode=smart&dispatchsample_ratio=1&workloads		
Hide Fields All Fields List Format 20 Per Page		
	Time	Event
>	5/22/26 12:03:32.336 PM	2026-05-22T12:03:32.336Z-04:00 cti-lab sshd(1641754): Disconnecting invalid user test 192.168.1.7 port 36446: Too many authentication failures (prow uth)
>	5/22/26 12:03:32.336 PM	host=cti-lab source=/var/log/authlog sourcetype=linux_secure
>	5/22/26 12:03:32.336 PM	2026-05-22T12:03:32.333Z-04:00 cti-lab sshd(1641754): error: maximum authentication attempts exceeded for invalid user test 192.168.1.7 port 36446 ssh2 (prow uth)
>	5/22/26 12:03:32.291 PM	host=cti-lab source=/var/log/authlog sourcetype=linux_secure
>	5/22/26 12:03:32.291 PM	2026-05-22T12:03:32.291Z-04:00 cti-lab sshd(1641752): PAM service(sshd) ignoring max retries: 6 > 3
>	5/22/26 12:03:32.291 PM	host=cti-lab source=/var/log/authlog sourcetype=linux_secure
>	5/22/26 12:03:32.291 PM	2026-05-22T12:03:32.291Z-04:00 cti-lab sshd(1641752): Disconnecting invalid user test 192.168.1.7 port 36446: Too many authentication failures (prow uth)
>	5/22/26 12:03:32.290 PM	host=cti-lab source=/var/log/authlog sourcetype=linux_secure
>	5/22/26 12:03:32.290 PM	2026-05-22T12:03:32.290Z-04:00 cti-lab sshd(1641752): error: maximum authentication attempts exceeded for invalid user test 192.168.1.7 port 36446 ssh2 (prow uth)
>	5/22/26 12:03:32.290 PM	host=cti-lab source=/var/log/authlog sourcetype=linux_secure
>	5/22/26 12:03:30.436 PM	2026-05-22T12:03:30.436Z-04:00 cti-lab sshd(1641763): Failed password for invalid user test 192.168.1.7 port 36446 ssh2
>	5/22/26 12:03:30.436 PM	host=cti-lab source=/var/log/authlog sourcetype=linux_secure
>	5/22/26 12:03:30.420 PM	2026-05-22T12:03:30.420Z-04:00 cti-lab sshd(1641761): Failed password for invalid user test 192.168.1.7 port 36456 ssh2
>	5/22/26 12:03:30.420 PM	host=cti-lab source=/var/log/authlog sourcetype=linux_secure
>	5/22/26 12:03:30.407 PM	2026-05-22T12:03:30.407Z-04:00 cti-lab sshd(1641754): Failed password for invalid user test 192.168.1.7 port 36446 ssh2
>	5/22/26 12:03:30.407 PM	host=cti-lab source=/var/log/authlog sourcetype=linux_secure

Figure 4: Splunk telemetry displaying authentication failures, invalid user attempts, and excessive login activity generated during the brute-force attack.

Adversary Simulation, Telemetry Validation & Investigation (continued)

```
Checking configuration... Done
Checking critical directories... Done
Checking indexes... Done
  validated: _audit _configtracker _internal _introspection _metrics _metrics_rollup _telemetry _theftbucket endpoint history main summary
Done
Checking filesystem compatibility... Done
Checking conf files for problems... Done
Checking default conf files for edits... Done
Validating installed files against hashes from '/home/mkent/splunk/splunk-9.1.2-b6b9c8185893-linux-2.6-x86_64-manifest'
All installed files intact.
Done
all preliminary checks passed.

Starting splunk server daemon (splunkd)...
PYTHONHTTPSVERIFY is set to 0 in splunk-launch.conf disabling certificate validation for the httplib and urllib libraries shipped with the embedded Python interpreter; must be set to "1" for increased security
Done

Waiting for web server at http://127.0.0.1:8000 to be available..... Done

If you get stuck, we're here to help.
Look for answers here: http://docs.splunk.com

The Splunk web interface is at http://ctl-lab:8000

mkent@ctl-lab:~$ cd ~/webtest
mkent@ctl-lab:~/webtest$ python3 -m http.server 8000 >> beacon.log 2>&1
mkent@ctl-lab:~/webtest$ cat beacon.log
192.168.1.7 - - [22/May/2026 21:50:01] "GET / HTTP/1.1" 200 -
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...

Keyboard interrupt received, exiting.
192.168.1.7 - - [22/May/2026 22:28:25] "GET / HTTP/1.1" 200 -
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

Figure 5: HTTP GET requests generated from the Kali attack system to a Python web service hosted on the Ubuntu target, simulating beaconing behavior.

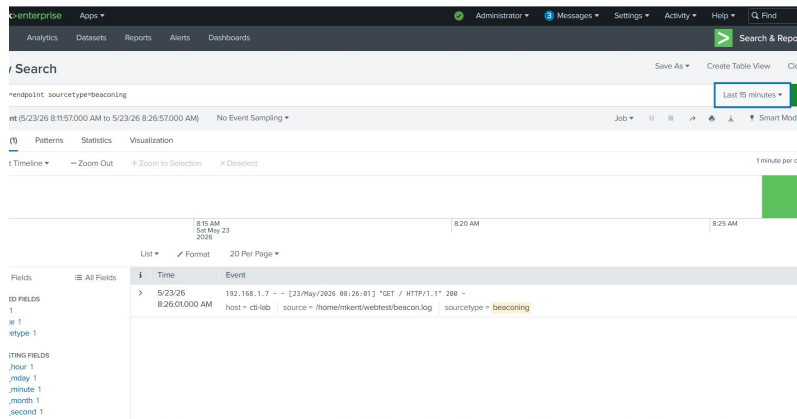


Figure 6: Splunk telemetry confirming successful ingestion of custom beaconing logs and visibility into HTTP communications.

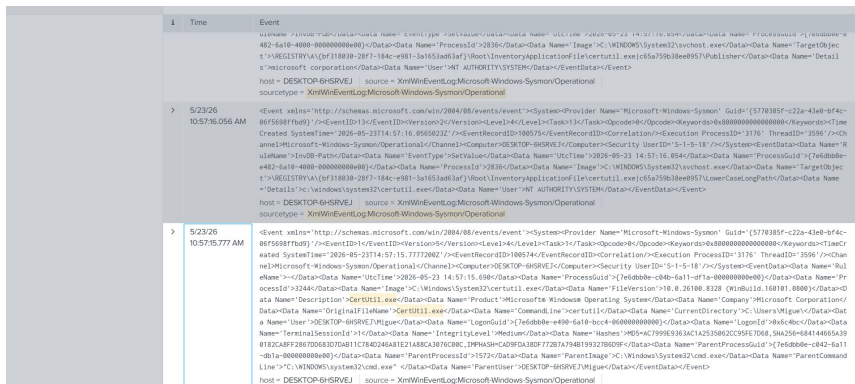


Figure 7: Sysmon Event ID 1 process creation event showing execution of Certutil from the Windows endpoint, including process metadata, user context, and parent-child process relationships.

Conclusion

Over the course of this project, a functional Security Operations & Detection Engineering Platform was developed to support telemetry collection, security monitoring, adversary simulation, and threat investigation across Windows and Linux environments.

Key capabilities implemented included centralized log collection through Splunk, endpoint visibility through Sysmon, Linux authentication monitoring, custom log ingestion, detection validation, and investigation workflows. These components were successfully integrated to provide visibility across multiple telemetry sources and support analyst-driven investigations.

Adversary simulations involving reconnaissance, SSH brute-force activity, HTTP beaconing, and LOLBin abuse generated security telemetry that was collected, analyzed, and investigated within Splunk. These exercises reinforced the importance of telemetry validation, data correlation, and visibility across both endpoint and network activity.

One of the most valuable outcomes of this project was gaining practical experience investigating activity across multiple log sources and understanding how security analysts use telemetry to identify, validate, and correlate attacker behavior.

With the completion of Part 1, the platform now provides a solid operational foundation for future enhancements including advanced detection engineering, threat hunting, automated response workflows, SOAR integrations, and AI-driven security operations.